Building LLMs from Scratch

Generated by Bookify

Table of Contents

LLM Fine-tuning for Classification

LLM Fine-tuning for Classification: Model Architecture

Fine-tuning a pre-trained Large Language Model (LLM) is crucial for adapting it to specific downstream tasks, such as text classification. This process leverages the extensive general language understanding acquired during pre-training and then specializes the model for a new objective. This section details the steps involved in preparing an LLM architecture for a binary text classification task, specifically identifying spam messages.

As introduced in prior sections, the initial phase of this project involved data acquisition and preprocessing. The SMS Spam Collection dataset [17] was downloaded, and its classes (spam and "ham" for non-spam) were balanced to ensure an equal number of samples (747 each) for robust training [1]. The text messages were then tokenized using a byte-pair encoder, padded to a uniform length of 120 tokens (based on the longest message in the dataset), and organized into input and target tensors for batch processing via data loaders [1]. With the data prepared, the next step is to configure and modify the LLM architecture.

1. Model Initialization with Pre-trained Weights

To avoid the immense computational cost and resources required for training an LLM from scratch, the process begins by initializing a GPT-like model architecture with pre-trained weights. OpenAI has made the weights for various GPT-2 models publicly available, allowing developers to utilize these powerful models as a foundation [2].

For this classification task, the GPT-2 small model, comprising 124 million parameters, is selected. The model's base configuration is set to match the parameters used during GPT-2's original pre-training to ensure compatibility with the loaded weights [3].

Table 1:

Configuration Parameter	Value	Description
Vocabulary Size	50257	Number of unique tokens the model can process.
Context Length	1024	Maximum sequence length the model can handle.
Dropout Rate	0	Dropout probability applied during training.
Query/Key/Value Bias	True	Indicates if bias terms are used in attention mechanisms.

The pre-trained GPT-2 parameters are downloaded and loaded into the custom GPT model architecture. This involves a utility function, `download_and_load_GPT2`, which retrieves the model's configuration settings and its entire parameter dictionary (including token embeddings, positional embeddings, transformer block parameters, and final normalization layer parameters) [4]. These parameters are then systematically loaded into the corresponding layers of the custom GPT model using a `load_weights_into_GPT` function [5]. This effectively "equips" the custom model with the language understanding capabilities of the pre-trained GPT-2.

To verify successful weight loading, the model can be tested for text generation. Providing an input prompt like "Every effort moves you" should result in a coherent and grammatically correct continuation, such as "forward. The first step is to understand the importance of your work" [6]. This confirms that the pre-trained parameters are functioning as expected for their original task of next-token prediction.

However, a pre-trained language model, even with its vast knowledge, does not inherently perform classification. An attempt to perform zero-shot classification by prompting the model with an instruction like "Is the following text spam? Answer with a yes or no: [spam message]" demonstrates its limitation. The model, without specific fine-tuning, struggles to follow such instructions and provide a direct "yes" or "no" answer, instead continuing the text generation task [7]. This highlights the necessity of fine-tuning for task-specific adaptation.

2. Modifying the Architecture for Classification

The core modification for classification fine-tuning involves adapting the model's output layer. In its original form, a GPT model's output layer (often referred to as the "output head") is a linear layer that maps the hidden state representation to a

probability distribution over the entire vocabulary (50257 tokens in this case) for next-token prediction [8].

For classification, this output head is replaced with a new linear layer, known as a **classification head**. This new layer maps the model's hidden state (with an embedding dimension of 768) to a smaller number of output units, corresponding to the number of classes in the classification task. For binary spam classification, this means mapping to two units: one for "spam" and one for "no spam" [9]. This approach is generalizable; for a multi-class classification task with N classes, the output head would map to N units.

The architectural change can be visualized as follows:

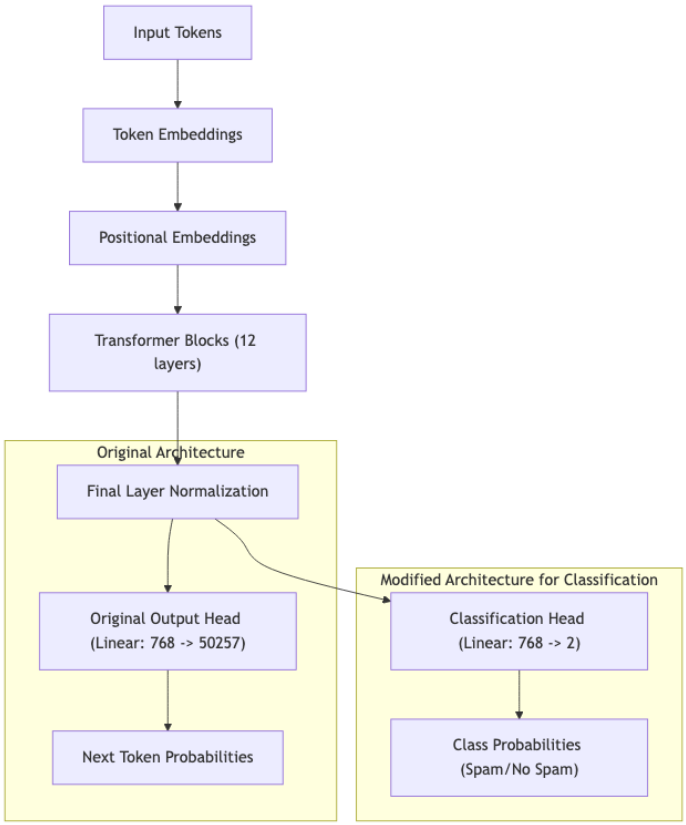

Figure 1:

To implement this, all parameters of the pre-trained model are initially "frozen" by setting their `requires_grad` attribute to `False`. This prevents them from being updated during the initial stages of fine-tuning [10]. Then, specific layers are "unfrozen" (or explicitly made trainable) to allow for adaptation to the classification task:

1. **Classification Head**: The `model.output_head` is replaced with a new `nn.Linear` layer that maps the hidden dimension (768) to the number of classes (2). Parameters of this new layer are trainable by default [11].

2. **Final Transformer Block**: The parameters of the last transformer block (`model.transformer_blocks[-1]`) are unfrozen. This allows the model's highest-level feature extraction capabilities to adapt to the nuances of the classification task [12].
3. **Final Layer Normalization**: The parameters of the `model.final_norm` layer, which connects the transformer blocks to the output head, are also unfrozen [13].

This selective fine-tuning strategy is efficient because the lower layers of the pre-trained LLM (such as token and positional embeddings, and earlier transformer blocks) have already learned fundamental language structures and semantics applicable across a wide range of tasks [14]. By only fine-tuning the upper layers and the newly added classification head, the model can quickly specialize for the new task without retraining its foundational language understanding.

When processing an input sequence, the model generates an output for each token. For classification, only the output corresponding to the **last token** in the sequence is typically used. This is because, due to the self-attention mechanism, the last token's representation has attended to and integrated information from all preceding tokens in the sequence, making it the most contextually rich representation for making a classification decision [15].

For example, if the input is "Do you have time?", the model will produce two output values (for "spam" and "no spam") for each of the four tokens. The output for the "time" token (the last token) is extracted and used for the final classification [16].

Summary

This section detailed the crucial steps in preparing an LLM for classification fine-tuning. It covered initializing the model with pre-trained GPT-2 weights, demonstrating the need for fine-tuning through a zero-shot attempt, and then modifying the architecture by replacing the original output head with a task-specific classification head. Furthermore, it explained the strategic decision to selectively unfreeze and train only the classification head, the final transformer block, and the final layer normalization, leveraging the pre-trained model's general language understanding while adapting it efficiently to the new task. The rationale for using the last output token for classification was also discussed.

With the model architecture now configured and ready, the next phase involves defining the loss function and evaluation metrics, which will enable the training process to begin.

References & Resources

LLM Fine-tuning for Classification

- https://drive.google.com/file/d/10Xbv0T9t74pEXejQ307w9ZABGLjHmFsu/view?usp=sharing
- https://archive.ics.uci.edu/dataset/228/sms+spam+collection

---

Footnotes

1. Video: *Coding the model architecture for LLM classification fine-tuning* @ 01:14
2. Video: *Coding the model architecture for LLM classification fine-tuning* @ 06:18
3. Video: *Coding the model architecture for LLM classification fine-tuning* @ 08:03
4. Video: *Coding the model architecture for LLM classification fine-tuning* @ 09:39
5. Video: *Coding the model architecture for LLM classification fine-tuning* @ 12:18
6. Video: *Coding the model architecture for LLM classification fine-tuning* @ 14:23
7. Video: *Coding the model architecture for LLM classification fine-tuning* @ 15:12
8. Video: *Coding the model architecture for LLM classification fine-tuning* @ 16:44
9. Video: *Coding the model architecture for LLM classification fine-tuning* @ 18:46
10. Video: *Coding the model architecture for LLM classification fine-tuning* @ 27:30
11. Video: *Coding the model architecture for LLM classification fine-tuning* @ 28:20
12. Video: *Coding the model architecture for LLM classification fine-tuning* @ 29:41
13. Video: *Coding the model architecture for LLM classification fine-tuning* @ 29:57
14. Video: *Coding the model architecture for LLM classification fine-tuning* @ 20:46
15. Video: *Coding the model architecture for LLM classification fine-tuning* @ 23:57
16. Video: *Coding the model architecture for LLM classification fine-tuning* @ 30:45
17. https://archive.ics.uci.edu/dataset/228/sms+spam+collection